

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Thursday 17th September, 2026

Chapter 1

Experiments

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on evaluating the method’s ability to consistently identify meaningful conceptual structures. The remainder of this chapter is organized as follows:

- Section 1.1 describes the software implementation of the CAR pipeline.
- Section 1.2 details the experimental configuration, including the environments, the reinforcement learning setup, and the training of the surrogate policies.
- Section 1.3 presents the training of the initial and surrogate policies and evaluates their behavioral fidelity.
- Section 1.4 analyzes the latent representations obtained by applying CAR and its metrics.
- Section 1.5 summarizes the findings and discusses their implications.

1.1 Software Implementation

For Deep Learning (DL) and Reinforcement Learning (RL), theoretical results are often difficult to apply to real-world cases. The effectiveness of methods in these fields is therefore primarily assessed through experimentation. Consequently, particular attention is given to the implementation of CAR. It relies on established libraries that provide efficient implementations of its main components. The code is made publicly available as open-source to support reproducible replication of the reported results.¹

CAR is organized as a pipeline of five stages, shown in Figure 1.1. Each stage is an independent executable, and stages communicate through artifacts written to disk. This allows each stage to be rerun or inspected independently. The parameters of each experiment are gathered in a single configuration object, described in Section 1.2. The following subsections describe the purpose of each stage, its implementation, and the libraries used.

¹https://github.com/Maxalaar/Clustering_Agent_Latent_Space

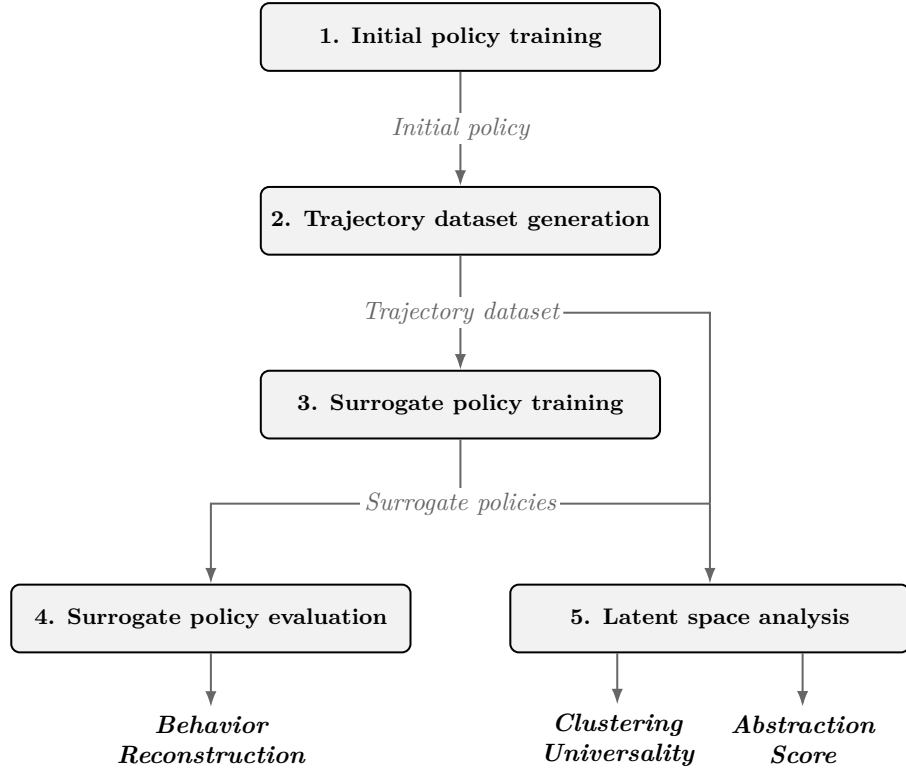


Figure 1.1: The CAR pipeline.

1.1.1 Initial policy training

The first stage trains the initial policy using RLlib [Moritz et al., 2018, Liang et al., 2018]. RLlib provides scalable implementations of RL algorithms and supports distributed training. The use of RLlib also allows CAR to be applied to environments implemented using the Gymnasium interface [Towers et al., 2024]. By default, the initial policy is trained using Proximal Policy Optimization (PPO) [Schulman et al., 2017]. Other RLlib algorithms, such as Asynchronous Proximal Policy Optimization (APPO) [Espeholt et al., 2018], Deep Q-Networks (DQN) [Mnih et al., 2013], and DreamerV3 [Hafner et al., 2023], can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [Paszke et al., 2019]. PyTorch is the standard deep learning frameworks and is widely used for developing and training neural networks. It provides GPU acceleration and supports a wide range of neural network architectures. This makes it well suited for implementing the different policy and critic architectures used in the experiments.

The resulting policy and critic parameters are saved as a checkpoint together with the environment and training configurations.

1.1.2 Trajectory dataset generation

The second stage builds the dataset used to train the surrogate policies. RLlib is used to collect trajectories in parallel using multiple environment runners. For each step, the dataset stores the observation together with the parameters of

the action distribution produced by the initial policy.

Trajectories are stored as Parquet files [Apache Arrow, 2016]. Its columnar structure allows efficient data access and supports concurrent processing. This is useful here because the datasets contain around one million observation–action pairs and cannot be loaded entirely into memory.

The resulting dataset is used both to train the surrogate policies and to generate the latent representations analyzed by CAR.

1.1.3 Surrogate policy training

The third stage trains the surrogate policies using behavior cloning on the trajectory dataset. This provides a supervised learning approach for replicating the initial policy. The training procedure is implemented using PyTorch Lightning [Falcon, 2019]. PyTorch Lightning provides a simplified interface for training neural networks with PyTorch. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution.

Each surrogate policy is saved as a checkpoint together with the configuration used for training.

1.1.4 Surrogate policy evaluation

The fourth stage evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy. Each surrogate is wrapped as an RLlib policy and evaluated in the original environment. Its mean cumulative reward over multiple episodes is compared with that of the initial policy. Evaluation episodes are collected using parallel environment runners.

This stage computes the *Behavior Reconstruction*. It therefore verifies that the surrogate policies reproduce the behavior of the initial policy sufficiently well before their latent representations are analyzed. If a surrogate does not meet the required level of behavior reconstruction, its training can be repeated.

1.1.5 Latent space analysis

The fifth stage performs the main analysis of CAR. The trajectory dataset and trained surrogate policies are used to generate the latent representations. These representations form the point clouds that are analyzed using clustering algorithms.

The clustering step relies on the cuML library [RAPIDS, 2023], which provides GPU-accelerated clustering algorithms. This reduces the computational cost and allows point clouds containing tens of thousands of representations to be analyzed.

The resulting partitions are used to compute the *Clustering Universality* and the *Abstraction Score*. The stage also produces the material required for the qualitative analysis, including the clustered point clouds and the cluster assignment of each observation.

Table 1.1 summarizes the main third-party libraries used in the pipeline.

Pipeline Stage	Library	Description
Initial policy training	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed RL library used to train the initial policy.
	PyTorch [Paszke et al., 2019]	Deep learning framework implementing the policy and critic networks.
	Gymnasium [Towers et al., 2024]	Common interface used to plug in any RL environment.
Trajectory dataset generation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Distributed environment runners used to collect trajectories in parallel.
	Apache Arrow [Apache Arrow, 2016]	Columnar file format (Parquet) used to store the trajectory datasets.
Surrogate policy training	PyTorch Lightning [Falcon, 2019]	Structured training loop used for the behavior cloning of the surrogate policies.
Surrogate policy evaluation	RLlib [Moritz et al., 2018, Liang et al., 2018]	Wraps each surrogate policy as an RL policy to evaluate it in the environment.
Latent space analysis	cuML [RAPIDS, 2023]	GPU-accelerated clustering algorithms used to analyze the latent representations.

Table 1.1: Main software libraries used at each stage of the CAR pipeline.

1.2 Experimental Configuration

This section describes the configuration of the experiments reported in this chapter. It covers the environments, the neural architecture of the policies, and the main parameters of each pipeline stage.

1.2.1 Environments

All experiments presented in this manuscript use environments from the Atari benchmark [Bellemare et al., 2013], accessed through the Arcade Learning Environment. The use of Atari environments offers several advantages. First, Atari is a widely used benchmark in the RL community, with many studies using these environments as standard evaluation settings. This makes the experimental results easier to compare with existing work. Second, these environments were not specifically designed for RL, which reduces the risk of introducing biases related to the design of the environments themselves. Finally, the benchmark provides a diverse set of well-known environments with varying levels of complexity. This diversity makes it possible to evaluate CAR across different types of tasks and levels of difficulty.

The configuration of the Atari environments follows common practice in RL research [Machado et al., 2017].

In each environment, the agent receives visual observations in the form of raw pixels. Each image is resized to 64×64 pixels and converted to grayscale to reduce memory requirements. To provide information about the dynamics of the environment, four consecutive observations are stacked along the channel dimension. The resulting observation has a shape of $64 \times 64 \times 4$. This allows the agent to infer the motion of objects from successive observations rather than relying on a single frame.

Frame skipping is also used to reduce the duration of each episode. The agent selects a new action every five frames, while the selected action is repeated for

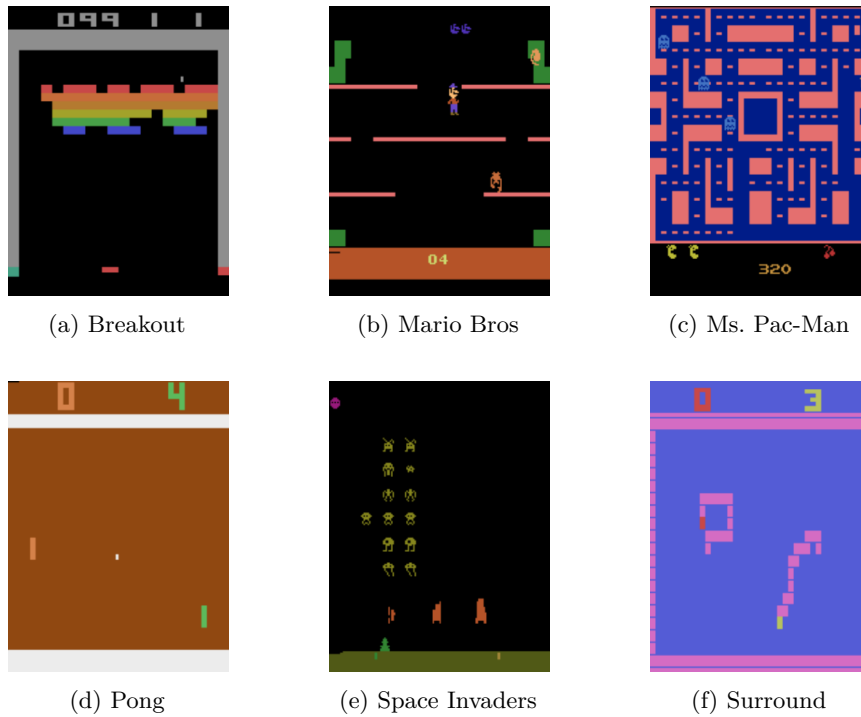


Figure 1.2: The six Atari environments used to evaluate CAR.

the intervening frames. This provides sufficient temporal resolution for effective control while reducing the number of decisions required during an episode.

The actions correspond to the different buttons available on the Atari controller. The resulting action space is therefore discrete. Since Atari games are fully deterministic, sticky actions are introduced to add stochasticity to the trajectories. With a probability of 0.05, the environment repeats the action selected at the previous step instead of applying the new action. This prevents the agent from simply memorizing a fixed sequence of actions during training.

With the environment configuration established, the objective of each environment is described below. Figure 1.2 provides an overview of the six environments used in the experiments.

Breakout. The agent moves a paddle at the bottom of the screen (Figure 1.2a) to bounce a ball into a wall of bricks, each broken brick scoring a point, and the ball speeding up as the wall is cleared.

Mario Bros. A single-screen platform game (Figure 1.2b). The player clears each phase by hitting enemies from below to flip them, then kicking them off the platforms before they recover. A central POW block flips every enemy at once but can only be used a few times per phase.

Ms. Pac-Man. The player moves through a maze (Figure 1.2c), eating pellets while avoiding four ghosts, with power pellets briefly making the ghosts edible. The maze structure and the many possible local situations, such as a nearby

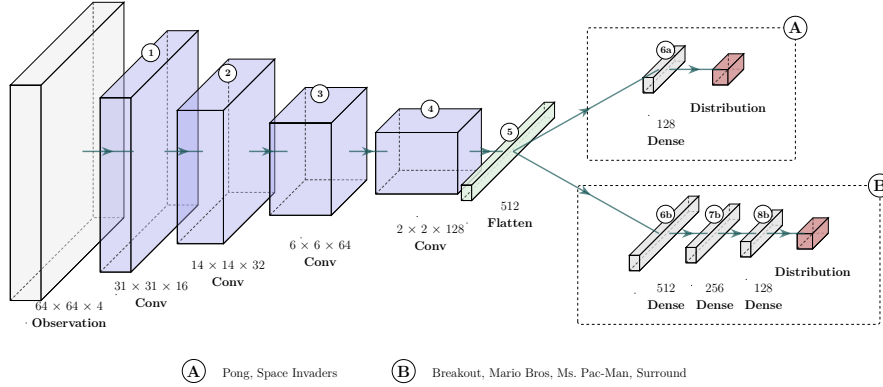


Figure 1.3: Architecture of the surrogate policies.

ghost, an intersection, or an active power pellet, produce a large number of distinguishable states.

Pong. A minimal two-player game (Figure 1.2d) in which the agent controls one paddle and must send the ball past the opponent.

Space Invaders. The agent moves a cannon along the bottom of the screen (Figure 1.2e), firing at descending rows of aliens while dodging their shots and using destructible shields, with a bonus saucer occasionally crossing the top.

Surround. A two-player game on a grid (Figure 1.2f). Each player leaves a solid trail behind a moving cursor, and the goal is to force the opponent to crash into a trail or a wall.

1.2.2 Neural architecture

The surrogate policies use a convolutional Neural Network (NN) to process the visual observations. The convolutional part consists of four layers with 16, 32, 64, and 128 channels, using 4×4 kernels with a stride of 2. Each convolutional layer is followed by a LeakyReLU activation. The resulting representation is then flattened and passed through a multilayer perceptron that produces the parameters of the action distribution.

The architecture of the multilayer perceptron varies across environments. For Pong and Space Invaders, it contains a single hidden layer with 128 units. For the other four environments, it contains three hidden layers with 512, 256, and 128 units. The size of the output layer matches the number of actions available in each environment: 4 for Breakout, 18 for Mario Bros, 9 for Ms. Pac-Man, 6 for Pong, 6 for Space Invaders, and 5 for Surround.

Figure 1.3 illustrates both configurations, together with the shape of the representation produced by each convolutional layer. Throughout the remainder of this chapter, the analyzed representations are numbered 1 to 5 for the four convolutional layers and the flattened representation, which are common to all six environments; the representations produced by the multilayer perceptron are then numbered 6a for Pong and Space Invaders (single hidden layer), or 6b, 7b, and 8b for the other four environments (three hidden layers).

Environment configuration	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Initial policy training	
Algorithm	PPO
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Batch size	2048
Optimizer	Adam
Learning rate	1×10^{-4}
Latent space analysis	
Embedding per surrogate policy	20 000
Clustering algorithm	k -means
Distance	Euclidean distance
Number of clusters K	2 to 30

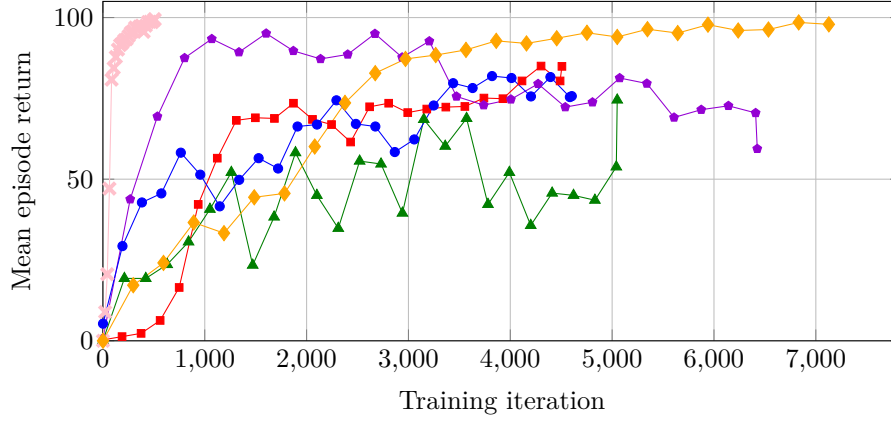
Table 1.2: Main experimental parameters.

1.2.3 Parameters

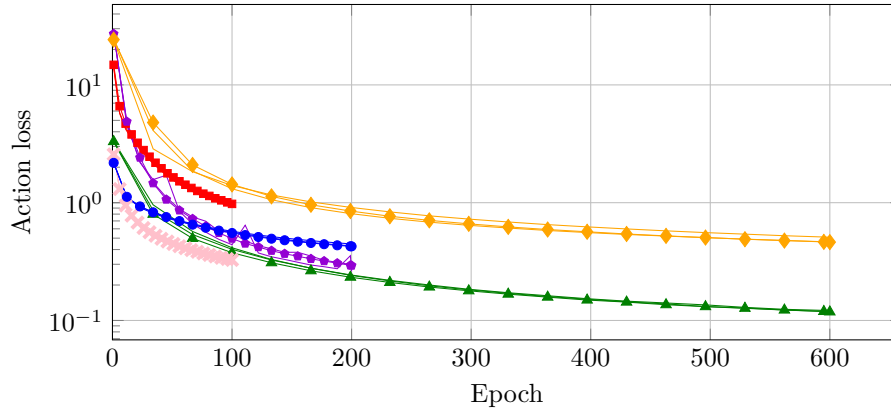
Having defined the environments and neural architectures, the remaining parameters of the CAR pipeline must now be specified. The main parameters shared across the experiments are summarized in Table 1.2. There are too many parameters to discuss them all in detail.

Given the large number of possible configurations, it is not feasible to evaluate all possible combinations. The choices made in this work are therefore based primarily on standard practices from the literature and on practical experience. Whenever possible, we used the simplest suitable configuration. For the clustering algorithm and distance metric in particular, the simplest suitable choices are used to establish a baseline.

These choices do not restrict the CAR methodology itself. Most components can be replaced or configured differently depending on the application. The evaluation metrics provide an objective basis for comparing these alternatives and determining which configurations produce the most consistent and meaningful results.



(a) Training of the initial policies with PPO.



(b) Training action loss of the surrogate policies.

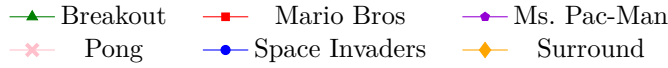


Figure 1.4: Training of the initial and surrogate policies.

1.3 Policy Training Analysis

This section reports the results of training the initial and surrogate policies for CAR, applied to the six environments. Section 1.3.1 verifies that the initial policies reach a competent level of play, while Section 1.3.2 evaluates whether the surrogate policies faithfully reproduce their behavior using the *Behavior Reconstruction*.

1.3.1 Initial policy training

Figure 1.4a shows how the initial policies improve during PPO training. For each environment, the mean episode return is normalized so that 0% and 100% correspond to the lowest and highest mean return observed during that run. In every environment the return increases and then reaches a plateau, at which point training is stopped. The number of iterations required to reach this plateau

Environment	<i>Behavior Reconstruction</i>
Breakout	0.83 $\pm$ 0.03
Mario Bros	0.99 $\pm$ 0.03
Ms. Pac-Man	1.00 $\pm$ 0.03
Pong	0.95 $\pm$ 0.01
Space Invaders	0.97 $\pm$ 0.07
Surround	1.00 $\pm$ 0.01

Table 1.3: Evaluation of *Behavior Reconstruction* across the six Atari environments.

varies from a few hundred, for Pong, to several thousand, for Surround and Ms. Pac-Man. For Ms. Pac-Man the return decreases after its peak. Since the best checkpoint is retained rather than the last one, this does not affect the policy that is subsequently analyzed.

To confirm that the trained policies play competently, their scores are compared with the reference scores reported by [Machado et al., 2017]. Each initial policy reaches a score of the same order as these references, so it constitutes a reasonable target for imitation.

1.3.2 Surrogate policy training

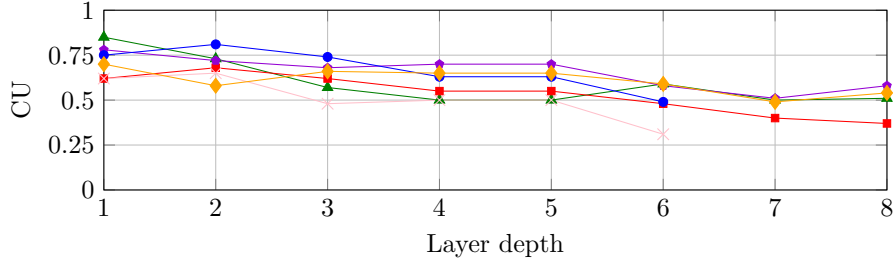
Figure 1.4b shows, for each environment, the training action loss of the three surrogate policies. In all cases, the loss decreases smoothly, and the three independently trained policies converge to the same value.

We further evaluate the ability of the surrogate policies to replicate the behavior of the initial policies using the *Behavior Reconstruction* (Table 1.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for Breakout to 1.00 for Ms. Pac-Man and Surround. They therefore reproduce the decisions of the initial policies closely enough for their latent representations to be analyzed in place of those of the initial policies.

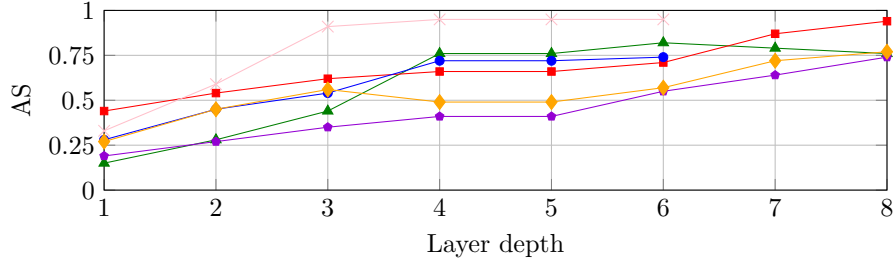
1.4 Latent Space Analysis

This section analyzes their Latent Space (LS) using *Clustering Universality* (CU), *Abstraction Score* (AS), and *Adjusted Clustering Universality* (ACU). For each environment and each layer, a random subset of 20,000 observations is embedded into the LS of each surrogate policy, and the resulting point cloud $\mathcal{E}$ is partitioned with k -means clustering [MacQueen, 1967] for every number of clusters $K \in [2, 20]$.

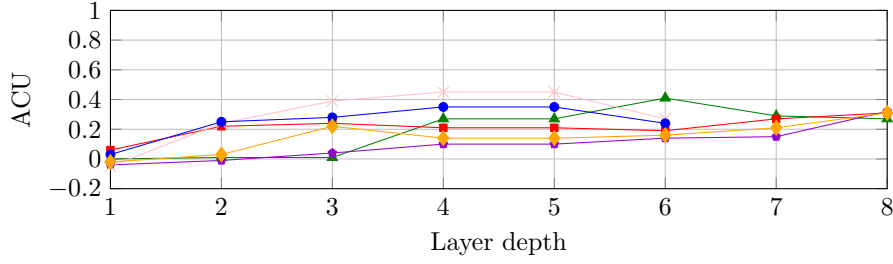
Figure 1.5 reports, for each environment, CU, AS, and ACU averaged over $K \in [2, 20]$ with respect to the depth of the analyzed layer. In every environment, CU tends to be highest in the earliest layers and to decrease with depth, while AS follows the opposite trend, increasing with depth as the representation becomes less tied to the raw observation, the action distribution, and the network’s initialization. ACU, which combines both effects, therefore increases with depth for Mario Bros, Ms. Pac-Man, and Surround, for which the last analyzed layer is



(a) CU with respect to the layer depth.



(b) AS with respect to the layer depth.



(c) ACU with respect to the layer depth.

▲ Breakout ■ Mario Bros ◆ Ms. Pac-Man
 ✕ Pong ● Space Invaders ◆ Surround

Figure 1.5: CU, AS, and ACU averaged over the number of clusters $K \in [2, 20]$, with respect to the depth of the analyzed layer, for each of the six environments.

the most informative, whereas it peaks at an intermediate depth and decreases again for Breakout, Pong, and Space Invaders.

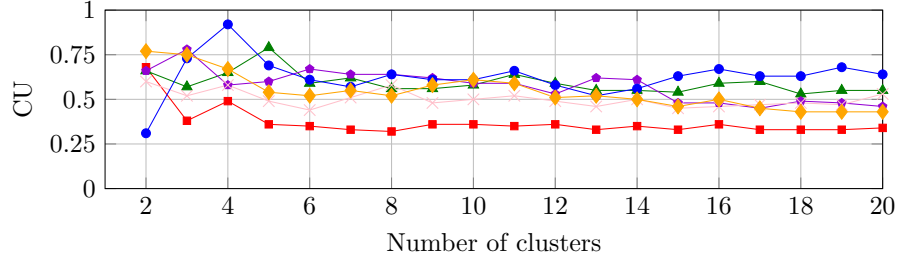
For each environment, the results are reported for the layer that achieves the highest average ACU across the tested values of K . This layer is therefore the one for which the agreement between independently trained surrogate policies is least explained by the observation space, the action space, or the network initialization. Table 1.4 reports this layer for each environment, together with the corresponding average CU, AS, and ACU over K ; the evolution of these three metrics with respect to K for that layer is then shown in Figure 1.6, and the full breakdown across every analyzed layer is reported in Appendix ??.

The best layer is the last analyzed one for Mario Bros, Ms. Pac-Man, and Surround, consistent with the increase of ACU with depth observed for these

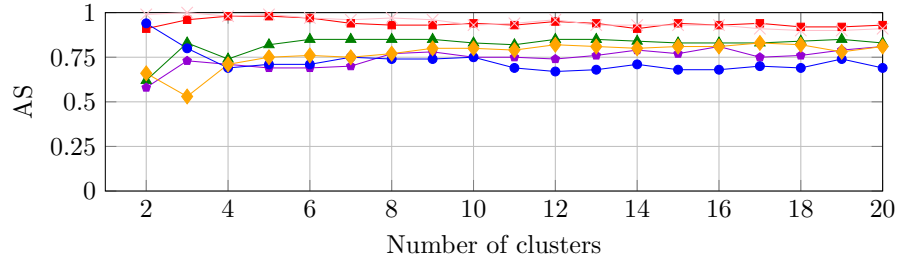
Environment	Layer	CU	AS	ACU
Breakout	6b	0.59 $\pm$ 0.06	0.82 $\pm$ 0.05	0.41 $\pm$ 0.06
Mario Bros	8b	0.37 $\pm$ 0.08	0.94 $\pm$ 0.02	0.31 $\pm$ 0.08
Ms. Pac-Man	8b	0.58 $\pm$ 0.08	0.74 $\pm$ 0.05	0.32 $\pm$ 0.07
Pong	4	0.50 $\pm$ 0.04	0.95 $\pm$ 0.03	0.45 $\pm$ 0.07
Space Invaders	4	0.63 $\pm$ 0.11	0.73 $\pm$ 0.06	0.35 $\pm$ 0.09
Surround	8b	0.54 $\pm$ 0.10	0.77 $\pm$ 0.07	0.31 $\pm$ 0.06

Table 1.4: Layer that maximizes the average ACU across $K \in [2, 20]$ for each environment, together with the corresponding average CU, AS, and ACU over K .

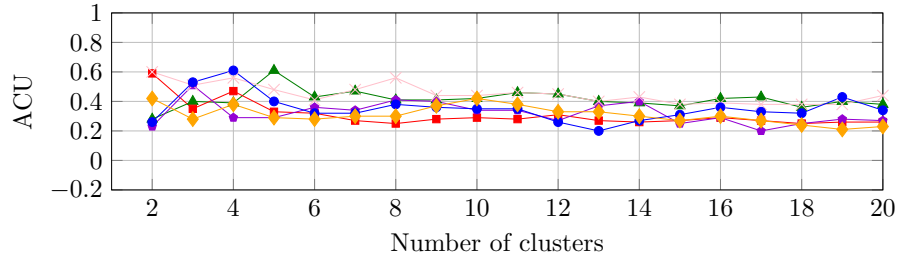
environments in Figure 1.5, whereas it is an intermediate layer for Breakout, Pong, and Space Invaders. Pong reaches the highest ACU (0.45 ± 0.07) among the six environments, combining a moderate CU with an AS that saturates from layer 3 onward, while Mario Bros combines the highest AS (0.94 ± 0.02) with the lowest CU (0.37 ± 0.08), yielding one of the lowest ACU values.



(a) CU with respect to the number of clusters.



(b) AS with respect to the number of clusters.



(c) ACU with respect to the number of clusters.

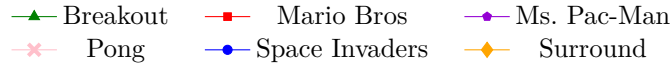


Figure 1.6: CU, AS, and ACU with respect to the number of clusters, for each of the six environments, for the layer that maximizes the average ACU in that environment.

1.5 Conclusion

The experiments reported in this chapter apply CAR and its metrics to five Atari environments. The surrogate policies reproduce the behavior of the initial policies with an average *Behavior Reconstruction* of 0.94 ± 0.02 . The latent clusterings are stable across independently trained surrogate policies, with a *Clustering Universality* of about 0.5 from three clusters onward, and they are largely independent of the observation and action spaces, with an average *Abstraction Score* of 0.99 ± 0.00 . Together, these results provide empirical support for the *Representation Convergence Hypothesis* (Hyp. ??) and, more broadly, for the hypothesis-driven approach underlying CAR.

Across all six environments, CU tends to be highest in the earliest layers

and to decrease with depth, while AS follows the opposite trend, increasing with depth as the representation becomes less tied to the raw observation, the action distribution, and the network’s initialization (Figure 1.5). ACU, which combines both effects, therefore peaks once the representation is abstract enough for AS to be high, but before CU has decayed too much. For Mario Bros, Ms. Pac-Man, and Surround, this optimum is reached at the last analyzed layer, suggesting that further gains might be observed by analyzing even deeper representations. For Breakout, Pong, and Space Invaders, in contrast, ACU peaks at an intermediate layer and decreases again toward the last one, indicating that the deepest representation is not always the most informative for CAR. This confirms that the layer at which the analysis is performed is not a neutral methodological choice and should itself be selected according to ACU rather than fixed a priori to the last convolutional layer, as was done in the single-layer analysis reported in earlier work.

Several other methodological choices have not yet been evaluated. These include the NN architecture, the distance used to compare embeddings, and the clustering algorithm. Their impact should be studied jointly to determine how much the discovered Concept Structure (CS) depends on the analysis procedure rather than on the policy itself.

The evaluation is also limited in scope. It considers only five Atari games, and the semantic labels attached to the clusters in Figure ?? were assigned by hand, which is time-consuming and subjective. Extending the study to more diverse environments and developing human-centered evaluation protocols would strengthen the validity of the resulting explanations.

Acronyms

286	ACU	<i>Adjusted Clustering Universality</i>	9–13
287	APPO	Asynchronous Proximal Policy Optimization	2
288	AS	<i>Abstraction Score</i>	9–13
289	CAR	Clustering Agent Representation	1–5, 7, 8, 12, 13
290	CS	Concept Structure	13
291	CU	<i>Clustering Universality</i>	9–13
292	DL	Deep Learning	1
293	DQN	Deep Q-Networks	2
294	GPU	Graphics Processing Unit	4
295	LS	Latent Space	9
296	NN	Neural Network	6, 13
297	PPO	Proximal Policy Optimization	2, 7, 8
298	RL	Reinforcement Learning	1, 2, 4

Bibliography

- [Apache Arrow, 2016] Apache Arrow (2016). Apache Arrow and Parquet. <https://arrow.apache.org>.
- [Bellemare et al., 2013] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. (2013). The arcade learning environment: An evaluation platform for general agents. Journal of Artificial Intelligence Research, 47:253–279. **Article.**
- [Espeholt et al., 2018] Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. (2018). IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Falcon, 2019] Falcon, W. (2019). PyTorch Lightning. <https://github.com/Lightning-AI/pytorch-lightning>.
- [Hafner et al., 2023] Hafner, D., Pasukonis, J., Ba, J., and Lillicrap, T. (2023). Mastering diverse domains through world models.
- [Liang et al., 2018] Liang, E., Liaw, R., Nishihara, R., Moritz, P., Fox, R., Goldberg, K., Gonzalez, J. E., Jordan, M. I., and Stoica, I. (2018). RLlib: Abstractions for distributed reinforcement learning. In Proceedings of the 35th International Conference on Machine Learning (ICML).
- [Machado et al., 2017] Machado, M. C., Bellemare, M. G., Talvitie, E., Veness, J., Hausknecht, M., and Bowling, M. (2017). Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. **Article. Web.**
- [MacQueen, 1967] MacQueen, J. (1967). Multivariate observations. In Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability, volume 1, pages 281–297. **Article.**
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Moritz et al., 2018] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., and Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. In 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pages 561–577.

- 334 [Paszke et al., 2019] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J.,
335 Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison,
336 A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy,
337 S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). PyTorch: An
338 imperative style, high-performance deep learning library. In Advances in
339 Neural Information Processing Systems (NeurIPS).
- 340 [RAPIDS, 2023] RAPIDS (2023). RAPIDS cuML: GPU-accelerated machine
341 learning. <https://github.com/rapidsai/cuml>.
- 342 [Schulman et al., 2017] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and
343 Klimov, O. (2017). Proximal policy optimization algorithms.
- 344 [Towers et al., 2024] Towers, M., Kwiatkowski, A., Terry, J., Balis, J. U.,
345 De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG,
346 A., et al. (2024). Gymnasium: A standard interface for reinforcement learning
347 environments.